

Implement Client Script & UI Policy (Incident)

Problem Statement

Incident records often require consistent and accurate data entry to ensure effective triage, routing, and resolution. However, relying solely on user awareness and manual checks can lead to incomplete, inconsistent, or incorrect data being submitted. This can impact reporting accuracy, SLA compliance, and overall service quality. To address this challenge, there is a need to enforce conditional field behaviour and validation directly at the user interface level. Implementing standardized UI Policies and Client Scripts helps control field visibility, mandatory status, and save-time validation, ensuring that Incident records meet defined business rules before they are submitted.

Objective

The objective of the Implement Client Script & UI Policy (Incident) project is to demonstrate how ServiceNow client-side controls can be used to enforce data integrity on Incident records. This implementation showcases how UI Policies and Client Scripts can dynamically make fields mandatory, auto-populate values, control field behavior, and prevent record submission when required conditions are not met. The solution ensures that Incidents are created with complete and valid information, improving data quality and consistency across the platform.

Skills

Incident Management
UI Policy
UI Policy Actions
Client Scripts
Form Validation

Task 1: Create UI Policy on Incident

Activity 1: Create UI Policy

1. Log in to the ServiceNow instance with administrative or configuration access.
2. Navigate to: **System UI → UI Policies**
3. Click **New** to create a new UI Policy record.
4. Fill in the following details:
 - **Name:** High Impact Control This name clearly represents the purpose of the policy, which is to apply additional controls when an Incident has a high impact.
 - **Table:** Incident Specifies that the UI Policy applies to Incident records.
 - **Active:** true Enables the UI Policy so it is enforced on the Incident form.

5. Configure the Conditions section as follows:

- **Field:** Impact
- **Operator:** is
- **Value:** 1 – High

6. This condition ensures the UI Policy is triggered only when the Incident impact is set to High.

7. And Ui policy action Assignment group

8. Check mandatory

9. Set **Reverse if false to true**. This ensures that when the condition is not met, any field changes applied by the policy are automatically reverted.

10. Review the configuration to ensure accuracy.

11. Click **Submit** to save the UI Policy.

Short description	Table	Conditions	Reverse if false	On load	Updated	Order
UI Policy to hide related feature flags when hidePagination set to true	UX List [sys_ux_list]	hide_pagination=true^EQ	true	true	2021-08-16 14:02:50	0
Mark fields as mandatory without DB override	Security tasks [sn_vsc_security_task]		false	true	2025-01-28 14:28:42	0
Always Hide Read-only Related List Field	Relationship [sys_relationship]	sys_idISNOTEMPTY^EQ	false	true	2015-08-21 17:28:31	1
Hide meta fields	label [sc_ic_question]		true	true	2014-01-21 12:43:35	1
Set Table column ReadOnly	To-dos Widget Mapping [sn_hr_sp_todos_widget_mapping]		true	true	2018-08-02 15:16:10	1
Clear save signature before action execution value - when precondition is not Signature	Function [sys_sg_button]	preconditions!=signature^EQ	true	true	2019-04-18 07:01:21	1
Content vs parameters	HTTP Method [sys_rest_message_fn]	http_method=get^ORhttp_method=delete^ORh...	true	true	2016-12-14 16:50:13	1
Instance Clone onChange Profile -	System Clone [clone_instance]	profileISEMPTY^EQ	true	true	2019-08-15	1

UI Policy - High Impact Control

Table: Incident [Incident]

Application: Global

Active: ☒

* Short description: High Impact Control

Order: 100

When to Apply | Script

Conditions: [Add Filter Condition](#) [Add OR Clause](#)

Impact is 1 - High

AND OR

If selected, the UI Policy applies to all form views; otherwise the UI Policy is view-specific

Apply the UI policy actions when the form is loaded and when the user changes values on the form

Global: ☒ On load: ☒

Reverse the effects of the UI policy actions when the Conditions evaluate to false

Tables that extend the specified Table inherit this UI Policy

Reverse if false: ☒ Inherit: ☐

UI Policy Action - New Record

UI policy actions specify exactly what actions to take on a specified field. The conditions specified in the UI policy determine when these actions are triggered. [More Info](#)

UI policy: High Impact Control

Table: Incident [Incident]

* Field name: Assignment group

Application: Global

Mandatory: True

Visible: Leave alone

Read only: False

Clear the field value: ☐

Submit

Task 2: Create UI Policy Action – Urgency

Activity 1:

Configure the Urgency Field

1. Open the UI Policy Actions

Go to the High Impact Control UI Policy you created earlier. Scroll down to the UI Policy Actions related list. This is where you define the specific behavior of fields when the policy conditions are met.

2. Create a New Action

Click New to create a UI Policy Action. This lets you configure how individual fields behave when the UI Policy is triggered.

3. Fill in the Action Details

- **Field name:** Urgency Select the Urgency field on the Incident form. This is the field you want to control.
- **Read-only:** true Check this option to make the Urgency field read-only whenever the UI Policy condition is met (Impact = High). Users will be able to see the field but cannot change its value.
- **Visible:** Leave this field as is (do not change). By leaving it unchanged, the field will remain visible on the form, but its behavior will be controlled by the Read-only setting.

4. Submit the Action

Click Submit to save the UI Policy Action. The action is now linked to the High Impact Control UI Policy and will automatically apply to the Urgency field when an Incident has High Impact.

5. Optional Verification To test the action:

- Open an Incident record.
- Set the Impact field to High.
- Check that the Urgency field becomes read-only.
- Change Impact to a different value to confirm that the field reverts back if needed.

UI policy actions specify exactly what actions to take on a specified field. The conditions specified in the UI policy determine when these actions are triggered. [More Info](#)

UI policy: High Impact Control

Table: Incident [Incident]

* Field name: Urgency

Application: Global

Mandatory: Leave alone

Visible: Leave alone

Read only: True

Clear the field value: ☐

Submit

Task 3: Create onChange Client Script

Activity 1: Create Client Script

1. Navigate to Client Scripts:

Go to System UI → Client Scripts.

2. Click New to create a new Client Script.

3. Fill in the details:

- **Name:** Auto set urgency for high impact – clearly shows the purpose.
- **Table:** Incident – the script runs only on Incident records.
- **Type:** onChange – executes when a field value changes.
- **Field name:** Impact – triggers the script when Impact is updated.
- **Active:** true – ensures the script runs immediately.

4. Paste the script in the script editor section.

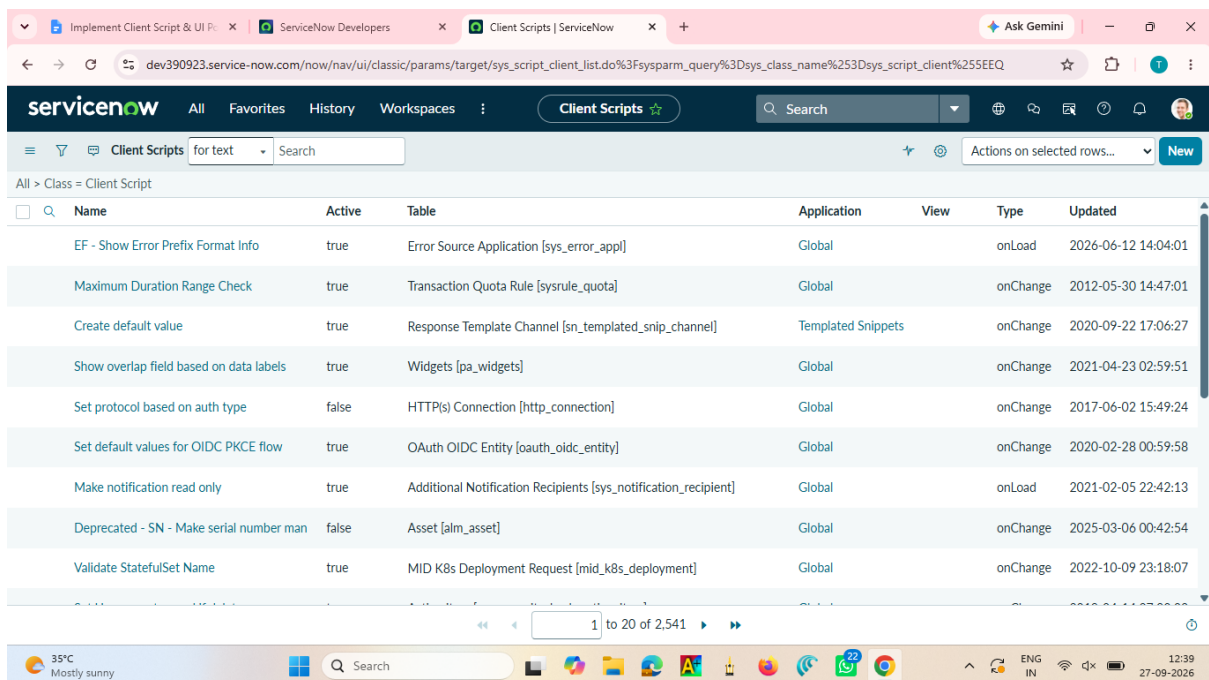
5. Click Submit to save the Client Script.

6. Verify: Open an Incident record and change the Impact field to High. The Urgency field should update automatically

Script:

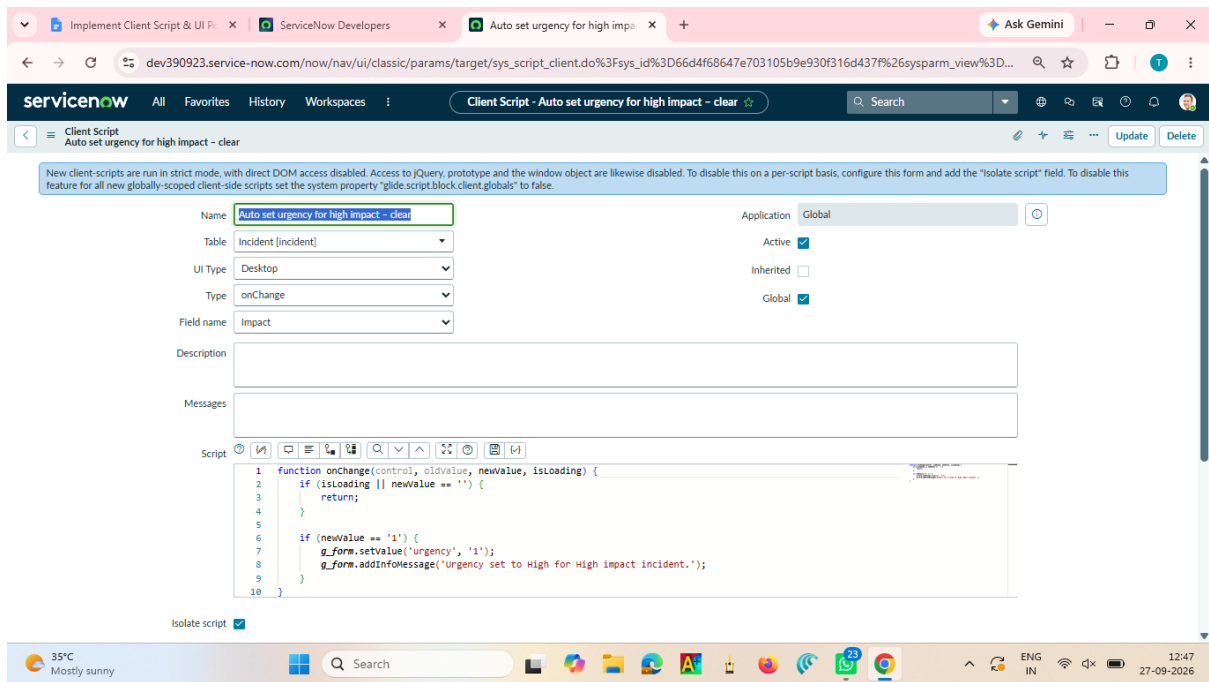
```
function onChange(control, oldValue, newValue, isLoading) {  
    if (isLoading || newValue === '') {  
        return;  
    }  
  
    if (newValue == '1') {  
        g_form.setValue('urgency', '1');  
        g_form.addInfoMessage('Urgency set to High for High impact incident.');    }  
}
```

5. Click Submit.



The screenshot shows the ServiceNow 'Client Scripts' interface. The table lists various scripts with columns for Name, Active status, Table, Application, View, Type, and Updated date. The table is filtered for 'Class = Client Script' and shows 20 of 2,541 items. The bottom of the screen shows a Windows taskbar with a search bar and system tray icons.

Name	Active	Table	Application	View	Type	Updated
EF - Show Error Prefix Format Info	true	Error Source Application [sys_error_appl]	Global		onLoad	2026-06-12 14:04:01
Maximum Duration Range Check	true	Transaction Quota Rule [sysrule_quota]	Global		onChange	2012-05-30 14:47:01
Create default value	true	Response Template Channel [sn_templated_snip_channel]	Templated Snippets		onChange	2020-09-22 17:06:27
Show overlap field based on data labels	true	Widgets [pa_widgets]	Global		onChange	2021-04-23 02:59:51
Set protocol based on auth type	false	HTTP(s) Connection [http_connection]	Global		onChange	2017-06-02 15:49:24
Set default values for OIDC PKCE flow	true	OAuth OIDC Entity [oauth_oidc_entity]	Global		onChange	2020-02-28 00:59:58
Make notification read only	true	Additional Notification Recipients [sys_notification_recipient]	Global		onLoad	2021-02-05 22:42:13
Deprecated - SN - Make serial number man	false	Asset [alm_asset]	Global		onChange	2025-03-06 00:42:54
Validate StatefulSet Name	true	MID K8s Deployment Request [mid_k8s_deployment]	Global		onChange	2022-10-09 23:18:07



Task 4: Create onSubmit Client Script (Save Validation)

Activity 1: Create Client Script

1. Navigate to Client Scripts:

Go to System UI → Client Scripts in your ServiceNow instance.

2. Click New to create a new Client Script.

3. Fill in the details:

- **Name:** Prevent save if Assigned To missing – clearly indicates that the script prevents saving if the field is empty.

- **Table:** Incident – applies the script only to Incident records.

- **Type:** onSubmit – executes when the user tries to save the record.

- **Active:** true – ensures the script is enforced immediately.

4. Paste the script

into the script editor section. This script should check if the Assigned To field is empty and prevent the record from saving if it is.

5. Click Submit

to save the Client Script.

6. Verify:

Try saving an Incident without filling the Assigned To field. The script should prevent the save and display a warning.

7. Paste the script:

```
function onSubmit() {  
    if (g_form.getValue('impact') == '1' &&  
        g_form.getValue('assigned_to') == "") {  
  
        g_form.showErrorBox(  
            'assigned_to',  
            'Assigned To is mandatory for High impact incidents.'  
        );  
  
        return false;  
    }  
  
    return true;  
}
```

8. Click **Submit**.

The screenshot shows the ServiceNow interface for configuring a Client Script. The browser tabs include 'Implement Client Script & UI Policies', 'ServiceNow Developers', and 'Auto set urgency for high impact'. The URL is 'dev390923.service-now.com/now/nav/ui/classic/params/target/sys_script_client.do%3Fsys_id%3D66d4f68647e703105b9e930f316d437f%26sysparm_view%3D...'. The page title is 'Client Script - Auto set urgency for high impact - clear'. A notification bar states: 'New client-scripts are run in strict mode, with direct DOM access disabled. Access to jQuery, prototype and the window object are likewise disabled. To disable this on a per-script basis, configure this form and add the "Isolate script" field. To disable this feature for all new globally-scoped client-side scripts set the system property "glide.script.block.client.globals" to false.'

The configuration form includes the following fields:

- Name: Auto set urgency for high impact - clear
- Table: Incident [Incident]
- UI Type: Desktop
- Type: onChange
- Field name: Impact
- Application: Global
- Active: ☒
- Inherited: ☐
- Global: ☒

Description and Messages fields are empty.

The Script editor contains the following code:

```
1 function onChange(control, oldValue, newValue, isLoading) {  
2     if (isLoading || newValue == '') {  
3         return;  
4     }  
5  
6     if (newValue == '1') {  
7         g_form.setValue('urgency', '1');  
8         g_form.addInfoMessage('Urgency set to High for High impact incident.');9     }  
10 }
```

The 'Isolate script' checkbox is checked.

The bottom of the screen shows a Windows taskbar with a weather widget (35°C, Mostly sunny), a search bar, and various application icons. The system clock shows 12:47 on 27-09-2026.

Task 5: Create onCellEdit Client Script

Activity 1: Create Client Script

1. Navigate to Client Scripts:

Go to System UI → Client Scripts in your ServiceNow instance.

2. Click **New** to create a new Client Script.

3. Fill in the details:

- **Name:** Prevent state change via list edit – clearly indicates that the script prevents users from changing the State field directly in a list.

- **Table:** Incident – applies the script only to Incident records.

- **Type:** onCellEdit – runs whenever a field is edited directly in a list view.

- **Field name:** State – the script will monitor changes to the State field.

- **Active:** true – ensures the script is enforced immediately.

4. **Paste the script** into the script editor section. The script should prevent unauthorized changes to the State field when editing from the list view.

5. **Click Submit** to save the Client Script.

6. **Verify:** Open an Incident list and try to change the State field directly in the list. The script should block the change as intended.

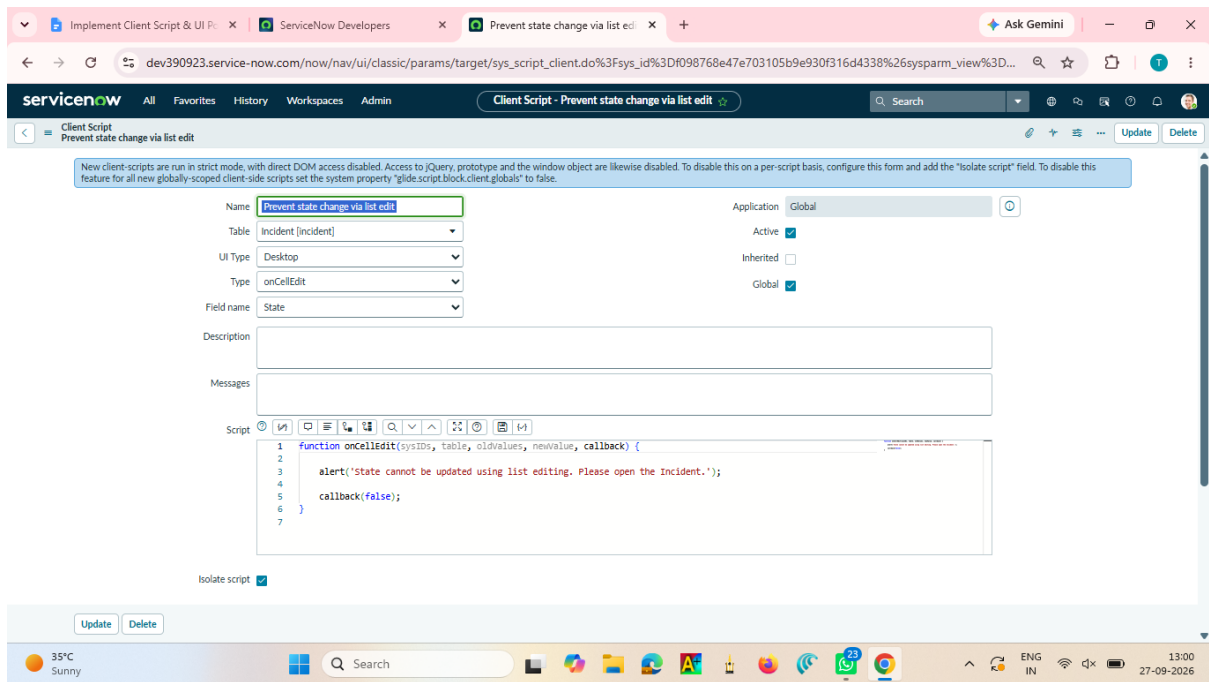
7. In the Script field, paste the following code:

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {  
    alert('State cannot be updated using list editing. Please open the Incident.');
```



```
    callback(false);  
}
```

8. **Click Submit.**



Activity1: Test Mandatory Enforcement

1. Navigate to Incident:

Go to Incident → Create New to open a new Incident form.

2. Set the Impact Field:

Select High for the Impact field. This will trigger the UI Policy and Client Script behaviors you configured earlier.

3. Leave Assigned To Empty:

Do not fill in the Assigned To field. This will test the onSubmit Client Script that prevents saving if Assigned To is missing.

4. Click Submit:

Attempt to save the Incident record.

5. Verify the Result:

- The Incident should not be saved because Assigned To is required when Impact is High.
- An error message should appear, indicating that the Assigned To field cannot be empty.

- Confirm that all UI Policy actions, such as read-only fields or automated urgency updates, are working as expected.

The screenshot shows the ServiceNow 'Incident - Create INC0010005' form. The form is partially filled with the following data:

- Number: INC0010005
- Caller: Adela Cervantsz
- Category: Inquiry / Help
- Subcategory: -- None --
- Service: (empty)
- Service offering: (empty)
- Configuration item: (empty)
- Channel: -- None --
- State: New
- Impact: 1 - High
- Urgency: 1 - High
- Priority: 1 - Critical
- Assignment group: CAB Approval
- Assigned to: (empty)
- Short description: testing ui policies
- Description: (empty)

Two error messages are displayed:

- A red message at the top: "The following mandatory fields are not filled in: Short description".
- A red message near the Assigned to field: "Assigned To is mandatory for High impact incidents."

The form has 'Submit' and 'Resolve' buttons at the top right. The bottom of the screen shows a Windows taskbar with the date 27-09-2026 and time 13:08.

Activity 2: Test Successful Save

1. Fill the Assigned To Field:

Open the Incident form and select a user in the Assigned To field. This ensures the onSubmit Client Script condition is satisfied.

2. Click Submit:

Attempt to save the Incident record.

3. Verify the Result:

- The Incident should be saved successfully without any errors.
- Confirm that all other configured behaviors, such as:
 - **Urgency auto-setting** (from the onChange Client Script)
 - **Read-only fields** (from the UI Policy Action)
 - **State restrictions** (from the onCellEdit Client Script) are functioning as expected

Activity 3: Reverse Condition Test

1. Open an Existing Incident:

Go to Incident → Open and select an Incident where Impact is currently set to High.

2. Change Impact:

Update the Impact field from High to Medium. This will test the Reverse if false functionality of your UI Policy and the behavior of your Client Scripy.

3. Verify the Field Behavior:

- **Assigned To** should no longer be mandatory.
- **Urgency** should now be editable, meaning the read-only UI Policy action is reversed.

4. Save the Record:

Click **Submit** to save the changes. The record should save successfully without errors, confirming that the reverse condition works as intended.

Activity 4: Test List Edit Blocking

1. Navigate to the Incident List:

Go to Incident → All to view all Incident records in a list view.

2. Attempt a List Edit

- Locate any Incident in the list.
- **Double-click the State field** to try editing it directly from the list.

3. Verify the Behavior:

- An **alert message** should appear, indicating that direct changes to the State field are not allowed.
- The **State value should remain unchanged**, confirming that the onCellEdit Client Script is working correctly.

Activity 5: Test Form-Based Update

1. Open the Incident Record:

Navigate to the same Incident record you used in previous tests.

2. Change the State:

On the Incident form, update the State field to a new value. This tests that changes via the form (not the list) are allowed.

3. Click Update:

Save the record by clicking Update.

4. Verify the Result:

- The State change should be saved successfully.
- Confirm that other fields, such as Assigned To and Urgency, still follow the configured UI Policy and Client Script rules.

The screenshot shows the ServiceNow Incident form for INC0010006. The 'Update' button is highlighted in the top right. A dropdown menu is open on the left, showing options like 'Save', 'Add to Visual Task Board', 'Copy Inc', 'Create Incident Task', 'Create Outage', 'Create Request', 'Create Child Incident', 'Refresh Impacted Services', 'Metrics Timeline', 'Follow on Live Feed', 'Analyze Access', 'Configure', 'Export', 'View', 'Create Favorite', 'Copy URL', 'Copy sys_id', 'Show XML', 'History', and 'Reload form'. The form fields include Caller (Adela Cervantz), Category (Inquiry / Help), Subcategory (None), Service (Service offering), Configuration item (testing ui policies), Description, Channel (None), State (In Progress), Impact (2 - Medium), Urgency (1 - High), Priority (2 - High), Assignment group, and Assigned to.

The screenshot shows the ServiceNow Incidents list view. The table displays the following data:

	Number	Opened	Short description	Caller	Priority	State	Category	Assignment group	Assigned to	Updated	Updated by
☐	INC0010006	2026-09-27 00:45:26	testing ui policies	Adela Cervantz	2 - High	In Progress	Inquiry / Help	(empty)	(empty)	2026-09-27 00:50:19	admin
☐	INC0010005	2026-09-27 00:35:06	testing ui policies	Adela Cervantz	1 - Critical	In Progress	Inquiry / Help	CAB Approval	Luke Wilson	2026-09-27 00:40:31	admin
☐	INC0010003	2026-09-27 00:22:06	practice	Abel Tuter	4 - Low	New	Inquiry / Help	(empty)	(empty)	2026-09-27 00:22:43	admin
☐	INC0009009	2018-08-30 01:06:16	Unable to access the shared folder.	David Miller	4 - Low	New	Inquiry / Help	(empty)	(empty)	2018-12-12 23:30:24	admin
☐	INC0009005	2018-08-31 21:35:21	Email server is down.	David Miller	1 - Critical	New	Software	(empty)	(empty)	2018-12-12 23:18:55	admin
☐	INC0009001	2018-09-11 20:56:26	Unable to post content on a Wiki page	David Miller	3 - Moderate	New	Inquiry / Help	(empty)	(empty)	2018-12-12 23:32:42	admin
☐	INC0008112	2019-07-29 11:48:43	Assessment : ATF Assessor	survey user	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2019-07-29 11:49:28	admin
☐	INC0008111	2019-07-22 14:04:57	ATF : Test1	System Administrator	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2019-07-22 14:05:48	admin
☐	INC0008001	2021-01-15 13:04:14	ATF:TEST2	survey user	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2021-01-21 15:31:42	admin
☐	INC0007002	2018-10-16 22:47:51	Need access to the common drive.	David Miller	4 - Low	New	Inquiry / Help	(empty)	(empty)	2018-12-12 23:28:49	admin
☐	INC0007001	2018-10-16 22:47:10	Employee payroll application server is down.	David Miller	1 - Critical	New	Hardware	Openspace	(empty)	2026-06-12 15:15:11	system
☐	INC0001990	2020-06-07 09:02:25	Unable to access the personal details section in payroll portal	Problem CoordinatorATF	5 - Planning	On Hold	Inquiry / Help	(empty)	(empty)	2020-06-07 09:04:34	admin
☐	INC0000059	2016-08-10 09:14:29	Unable to access team file share	Rick Berzle	New	New	Inquiry / Help	(empty)	(empty)	2016-08-10 09:14:29	admin
☐	INC0000058	2016-08-10 09:37:45	Performance problems with email	Bow Ruggeri	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2016-08-10 09:37:45	admin
☐	INC0000057	2016-08-10 09:14:59	Performance problems with wifi	Bertie Luby	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2016-08-10 09:14:59	admin
☐	INC0000055	2026-05-21 12:47:23	SAP Sales app is not accessible	Carol Coughlin	1 - Critical	In Progress	Service Desk	Service Desk	Beth Anglin	2026-06-04 16:08:31	admin
☐	INC0000054	2015-11-02 12:49:08	SAP Materials Management is slow or there is an outage	Christen Mitchell	1 - Critical	On Hold	Software	Service Desk	(empty)	2015-11-24 07:47:36	admin
☐	INC0000053	2026-05-21 13:48:46	The SAP HR application is not accessible	Margaret Grey	1 - Critical	In Progress	Inquiry / Help	Software	Beth Anglin	2026-06-11 12:42:25	admin
☐	INC0000052	2026-05-21 13:48:40	SAP Financial Accounting application appears to be down	Bud Richman	1 - Critical	In Progress	Software	Software	Fred Luddy	2026-06-04 14:50:24	system
☐	INC0000051	2026-05-21 13:48:32	Manager can't access SAP Controlling application	Joe Employee	1 - Critical	In Progress	Software	Software	Don Goodliffe	2026-06-11 12:40:45	admin

Conclusion

The Implement Client Script & UI Policy (Incident) micro project effectively demonstrates how UI Policies and Client Scripts can work together to enforce dynamic field behavior, automate updates, and prevent incorrect data submission on Incident forms. By using onChange, onSubmit, and onCellEdit scripts alongside UI Policy actions, the project ensures fields like Assigned To and Urgency respond appropriately to changes in Impact, maintaining clean and consistent data. This solution not only enforces mandatory fields and controls user interactions but also remains lightweight, efficient, and easy to implement, making it ideal for short-duration micro projects while showcasing practical best practices for improving form usability and data integrity in ServiceNow.